# Genetic programming for drone swarm behaviour optimisation

Ulysse Bourdic-Girard
*Ecole Nationale de l'Aviation Civile*
Toulouse, France
ulysse.bourdic-girard@alumni.enac.fr

Guillaume Claudon
*Ecole Nationale de l'Aviation Civile*
Toulouse, France
guillaume.claudon@alumni.enac.fr

***Abstract*—This study presents a method of leveraging genetic programming principles to determine the flight parameters of a drone swarm controlled by a bio-inspired algorithm developed by Matthieu Verdoucq. The study explores how this method can be used to optimise the behaviour of a drone swarm in a search-and-rescue scenario, heavily relying on parallelized GPU simulations via PyTorch.**

***Index Terms*—Collective motion, Genetic programming, 3D Flocking Algorithm, Distributed Control, Drone swarm, Unmanned Aerial Vehicle (UAV)**

## I. Introduction

Single unmanned aerial vehicles (UAVs) have proven to be exceptionally effective at a wide range of tasks, ranging from cinematography to pesticide application. Swarms of UAVs, although not yet commonly used, hold great promise for future applications. One of these is search and rescue (SAR). In such scenarios a drone swarm has an evident numerical advantage over a single UAV. Controlling the swarm and ensuring it efficiently covers the search area is however a significant challenge.

Our study is based on the bio-inspired flocking algorithm developed by Matthieu Verdoucq for his Ph.D. thesis [1]. A swarm controlled by this algorithm can exhibit emergent behaviours such as schooling (the drones flock together in an orderly fashion), swarming (the swarm remains tight-knit but the drones move erratically), or milling (the drones rotate around swarm's centre in an orderly fashion). Matthieu Verdoucq and his colleagues [2] have demonstrated these behaviours emerge from a specific set of parameters which define the equations used to control the drones.

Our goal is to implement genetic programming paradigms to optimise this set of parameters to adapt the swarm's behaviour to a SAR scenario.

This scenario is modeled by dividing the area to be explored into a grid. The gathering of information is simulated by assigning a numerical freshness value to each cell, which decreases over time.

This study was produced as part of the PIR-Drones minor at ENAC.

## II. Related Work

The work presented in this article is a continuation of the research done by Matthieu Verdoucq for his thesis, which he successfully defended in February 2024.

### A. Flocking algorithm

The flocking algorithm developed by Matthieu Verdoucq [1] is inspired by the behaviour of a school of Rummy-nose tetra (*H. Rhodostomus*). Tamás Vicsek and Anna Zafeiris [3] were able to extract a model of social interactions between the fish which leads to emergent behaviour. Matthieu Verdoucq then adapted this model to the dynamics of drones by adding, amongst other things, a vertical component to the social interactions.

Lei et al. [4] have shown that Rummy-nose tetra only interact with a limited number of their neighbours. Matthieu Verdoucq implemented this in his interaction model which means the drones need only gather a limited amount of information from their social environment to display coordinated behaviour.

The emergent behaviour of the swarm is influenced by a set of parameters which determine the guidance of each individual drone in the swarm. Verdoucq et al. [5] have shown how changing these parameters can alter the behaviour of the swarm in real-time.

These studies have observed that certain combinations of parameters induce the same behaviour in the swarm. Each of these specific behaviours is called a phase. The research produced by Matthieu Verdoucq for his thesis principally focuses on phases emerging from interactions between the alignment and attraction coefficients $\gamma_{\text{ali}}$ and $\gamma_{\text{att}}$. Matthieu Verdoucq [1] has identified three distinct phases linked to these parameters: schooling, swarming and milling.

This means that although the paths of individual UAVs in the swarm cannot be controlled, their global behaviour can. However, there are many other parameters which influence the behaviour of the swarm which were not studied by Matthieu Verdoucq. The complexity of the interactions between these parameters prevents a manual tuning of this set of parameters. Genetic programming paradigms have been used to optimise drone swarm objectives and are

promising means of accomplishing our objective: optimising the set of drone guidance parameters to a specific task.

### B. Genetic programming

Genetic programming algorithms are powerful because they do not require knowing the structure of the solution to find it [6]. The problem at hand: determining the right parameters to induce the desired behaviour in the drone swarm, is therefore very well suited to genetic programming because the interactions between the parameters and their resulting influence on the swarm's behaviour are opaque.

While some studies, such as the work by Sefa Abraç and Tarik Veli Mumcu [7], focus on combinatorial mutation strategies like displacement or scramble mutations for task assignment, optimizing flight dynamics requires a continuous parameter space approach. Our study will therefore adjust the real-valued parameters controlling the drone swarm's behaviour by employing continuous real-parameter optimization techniques. Specifically, we will utilize uniform crossover operations combined with Gaussian noise mutations to ensure both the exploitation of successful traits and the exploration of new behavioural paradigms.

Another approach consists of using genetic programming to autonomously design the high-level logical architecture of a swarm. For instance, the GEESE-BT algorithm developed by Neupane and Goodrich [8] utilizes multi-agent Grammatical Evolution to map genetic binary strings directly into functional, executable Behavior Trees. By embedding primitive swarm behaviors within a context-free grammar, their framework successfully evolves decentralized, discrete action controllers for complex spatial tasks like foraging and nest maintenance without requiring manually engineered state machines. However, while this structural evolution is highly effective for high-level spatial decision-making, it fundamentally operates in a discrete search space rather than the continuous domain needed for precise physical maneuvering.

### C. Information gathering strategies

A genetic algorithm can only be as good as its fitness function. In the cas of a SAR scenario the fitness depends a lot on how the gathered information is shared between the drones in the swarm.

One such method is pheromone-based stygmergic coordination [9], [10]. This method relies on digital pheromones being released by the drones to mark the cells they have explored. These pheromones decay and disappear over time, representing the information losing relevance as time goes on. The pheromones repel other drones, thus encouraging the swarm to fly to cells which have yet to be explored. Xuan et al. [10] have even explored how the use of two distinct pheromones in an under water exploration scenario: a short term pheromone to encode the recent activity of the swarm and a long term pheromone to encode the global exploration coverage. They have showed this method can achieve a very high coverage uniformity of the water column.

## III. Our approach to genetic programming

The proposed optimization framework utilizes a heavily parallelized, tensor-based Genetic Algorithm (GA) implemented in PyTorch. This architecture allows for the simultaneous evaluation of thousands of potential swarm configurations by fully leveraging GPU acceleration. Thanks to this deep tensorization, the memory footprint is extremely well-managed; evaluating a massive population of 5000 individuals requires minimal VRAM overhead (typically remaining well under 4 GB), as the entire physics and cost simulation is processed via unified matrix operations rather than sequential loops. Furthermore, to ensure strict reproducibility of the results and deterministic execution, the framework relies on fixed manual seeding (e.g., `torch.manual_seed()`) for all random number generation across the initialization and evaluation phases. At the start of the optimization, the initial population of genes is generated by sampling these parameters from uniform random distributions within predefined, physically plausible bounds.

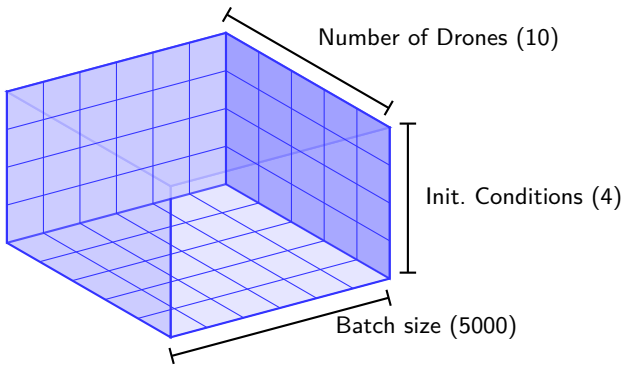



Fig. 1. 3D Tensor structure used for parallel swarm evaluation in PyTorch.

During the evaluation phase, each genome dictates the behavior of a simulated drone swarm over a fixed number of integration steps. The fitness of each individual is evaluated using a composite cost function that aggregates multiple behavioral metrics into a single score (see Section IV). This function penalizes undesirable flight characteristics—such as excessive turning efforts or inter-drone collisions—while rewarding overarching mission objectives like spatial coverage or collective alignment. By blending these diverse and sometimes conflicting factors, the cost function establishes a complex fitness landscape.

To ensure the resulting swarm behaviors are robust and not merely overfitted to a specific starting layout, the algorithm evaluates each genome across multiple distinct initial spatial conditions. The framework can then assign a final fitness score using one of three evaluation strategies: "average", "best", or "minmax". The "average" strategy

computes the mean cost across all environments, promoting generally good behavior but potentially masking edge-case failures. The "best" strategy isolates the minimum cost achieved, rewarding peak performance but risking fragility in non-ideal scenarios. Conversely, the implemented "min-max" strategy records the costs from all starting conditions and assigns the maximum—or worst—cost as the genome's definitive score. By actively optimizing for this worst-case scenario, the algorithm forces the selection of universally resilient configurations, significantly narrowing the reality gap for potential real-world deployment.

Following evaluation, the algorithm applies evolutionary operators to generate the next iteration of parameters. It utilizes a tournament selection mechanism with a tournament size of 3 to choose parent genomes, randomly pitting three individuals against each other and selecting the one with the lowest worst-case cost. A uniform crossover operator then blends the genes of these selected parents. To maintain genetic diversity and prevent premature convergence, a double mutation strategy is applied. First, a local mutation introduces a Gaussian scaling noise ($X \sim \mathcal{N}(1.0, 0.3^2)$), with a mutation probability $p_m(g)$ that decays as generations ($g$) progress. This allows for broad parameter search initially and fine-tuning in later stages:

$$p_m(g) = 0.35 - 0.20\left(\frac{g}{G}\right) \tag{1}$$

Second, a global mutation operator introduces a large parameter jump that completely replaces genes with new random values, applied with a fixed probability of 5% across the entire generation. This ensures the population can abruptly escape local minima in the complex, non-linear fitness landscape. Finally, a strict elitism mechanism is enforced: the single absolute best-performing parameter set is preserved entirely untouched and injected directly into the first slot of the subsequent generation, securing monotonic optimization progress throughout the training cycle.

## IV. Emergence of Collective Behaviors: Milling

### A. Defining the Cost Function

To actively search for specific topological phases, the fitness function evaluated by the Genetic Algorithm must explicitly reward desired macroscopic states while penalizing collisions and excessive energy consumption. We define a multi-objective cost function balancing group polarization, milling momentum, and control effort.

- $N$: number of drones
- $p_i = (x_i, y_i)$: position
- $\phi_i$: heading
- $v_i$: speed

The cost of the effort cost $c_{\text{effort}}$ is calculated as follows:

$$c_{\text{effort}} = \sum_{i=1}^{N} |\dot{\phi}_i| \tag{2}$$

The center of the swarm $B$ is determined:

$$B = \frac{1}{N}\sum_{i=1}^{N} p_i \tag{3}$$

The distance of each drone to this center is calculated as such:

$$d_i = \|p_i - B\|_2 \tag{4}$$

The cost of the dispersion cost $c_{\text{disp}}$ is calculated as follows:

$$c_{\text{disp}} = \left| \left( \frac{1}{N}\sum_{i=1}^{N} d_i \right) - 5.0 \right| \tag{5}$$

The polarisation parameter is determined as follows:

$$P = \sqrt{\left( \frac{1}{N}\sum_{i=1}^{N} \cos\phi_i \right)^2 + \left( \frac{1}{N}\sum_{i=1}^{N} \sin\phi_i \right)^2} \tag{6}$$

This allows for the calculation of the polarisation cost $c_{\text{pol}}$:

$$c_{\text{pol}} = (1 - P) \tag{7}$$

To calculate the collision cost $c_{\text{coll}}$, we first determine the distance between drones:

$$d_{ij} = \|p_i - p_j\|_2 \tag{8}$$

Thus:

$$c_{\text{coll}} = \frac{1}{2}\sum_{i=1}^{N}\sum_{\substack{j=1 \\ j\neq i}}^{N} \mathbb{1}_{(d_{ij} < d_{\text{coll}})} \tag{9}$$

To compute the milling cost $c_{\text{mill}}$, we first compute the speed vector $V_i$ for each drone:

$$V_i = \begin{pmatrix} v_i \cos\phi_i \\ v_i \sin\phi_i \end{pmatrix} \tag{10}$$

Then we determine the average speed of the swarm:

$$\bar{V} = \frac{1}{N}\sum_{i=1}^{N} V_i \tag{11}$$

Thus follows the centred speed:

$$\Delta v_i = v_i - \bar{V} \tag{12}$$

And the centred position:

$$\Delta p_i = p_i - B \tag{13}$$

These are used to determine the angle of the postion to the swarm's centre:

$$\theta_i = \text{atan2}(\Delta p_{i,y}, \Delta p_{i,x}) \tag{14}$$

And the angle of the speed vector to the swarm's centre:

$$\phi_{v,i} = \text{atan2}(\Delta v_{i,y}, \Delta v_{i,x}) \tag{15}$$

These then allow for the computation of the milling cost:

$$c_{\text{mill}} = \left| \frac{1}{N}\sum_{i=1}^{N} \sin(\theta_i - \phi_{v,i}) \right| \tag{16}$$

To ensure the stability of the emergent milling phases and prevent the swarm from drifting—a behavior frequently observed during the optimization of unconstrained circular motion—we introduce a stationary cost $c_{\text{stationary}}$. This cost penalizes the velocity of the swarm's barycenter:

$$c_{\text{stationary}} = \|\bar{V}\|_2 \tag{17}$$

The final cost $c_{\text{tot}}$ function is the sum of all of these costs, to which the exploration cost can be added if relevant (this cost will be detailed in a further section). Each cost $c_x$ is associated to a weight $W_x$ which is used to tune the relative influences of these parameters in the final cost function:

$$\begin{aligned} c_{\text{tot}} = {} & c_{\text{effort}} \times W_{\text{effort}} \\ & + c_{\text{disp}} \times W_{\text{disp}} \\ & + c_{\text{pol}} \times W_{\text{pol}} \\ & + c_{\text{coll}} \times W_{\text{coll}} \\ & + c_{\text{mill}} \times W_{\text{mill}} \\ & + c_{\text{stationary}} \times W_{\text{stationary}} \\ & + c_{\text{explo}} \times W_{\text{explo}} \end{aligned} \tag{18}$$

With only a pure milling weight on the cost function, by minimizing $c_{\text{tot}}$, the Genetic Algorithm successfully identified regions in the parameter space that organically lead to a stable milling formation. Figure 2 illustrates a typical simulation output where the swarm converges into a continuous vortex, demonstrating the efficacy of our multi-objective cost function in isolating this specific topological phase.

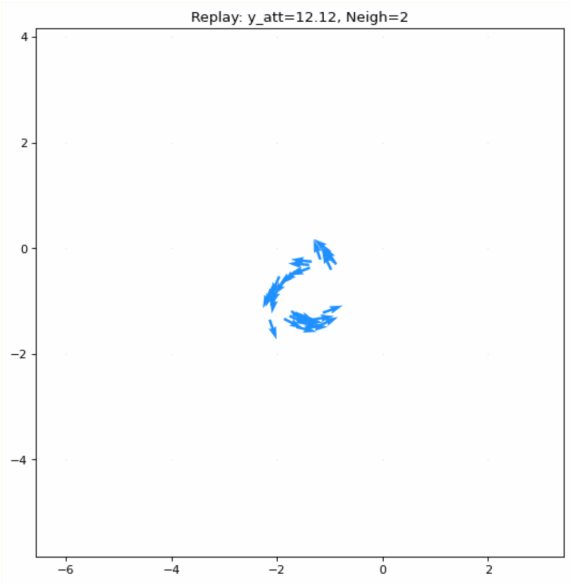



Fig. 2. Visual representation of a successful milling phase emerged from the GA optimization. The drones (blue vectors) exhibit stable circular motion around the swarm's barycenter while maintaining strict collision avoidance constraints.

### B. Parameter Sweep and Scalability

The emergence of a stable milling phase is notoriously difficult to achieve through manual tuning due to the highly non-linear dynamics and cascading interactions between the bio-inspired control parameters. Therefore, successfully isolating the parameter manifold that consistently triggers milling serves as a fundamental proof of concept for our Genetic Algorithm (GA). It demonstrates the algorithm's capability to navigate a complex, multi-dimensional continuous search space to optimize macroscopic swarm behaviour before applying it to more complex operational scenarios.

We initially employed the GA to discover parameter bounds that reliably trigger the milling phase for a fixed population. However, a set of parameters optimized for $N = 10$ drones may degrade, shear, or cause the vortex to collapse entirely when applied to $N = 30$. To ensure the robustness of our model, we conducted a systematic parameter sweep across varying swarm sizes ($N \in \{10, 15, 20, 25, 30\}$) to analyze the scalability of the discovered behavioral regions. To avoid statistical bias from generational cloning, only the absolute best, unique individual from each optimization run was extracted for analysis.

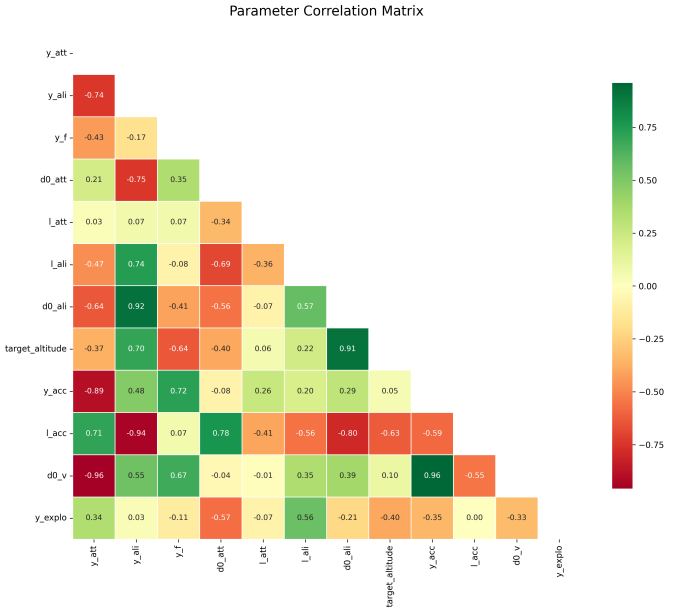



Fig. 3. Parameter correlation matrix generated from the unique global optimums across all tested swarm sizes ($N = 10$ to 30). The diverging color scale highlights the true macroscopic scaling laws and physical trade-offs required to sustain the milling phase at any scale.

The resulting correlation matrix (Fig. 3) filters the dataset to include only the active genes, revealing the underlying physical rules the GA had to discover to maintain the vortex formation at scale. The matrix highlights three critical dynamic dependencies:

- **Kinematic Synchronization Scaling ($r = 0.96$):** A near-perfect positive correlation exists between the

social acceleration multiplier ($y_{\text{acc}}$) and the nominal velocity distance ($d0_{\text{v}}$). The algorithm discovered a fundamental scaling law for speed synchronization: as the swarm's scale changes, a larger zone of speed influence must be compensated by a proportionally stronger acceleration authority. This ensures that agents on the outer edge of a larger milling ring can accelerate rapidly enough to keep pace with the inner agents.

- **Alignment Field Proportionality ($r = 0.92$):** The matrix shows a very strong positive correlation between the alignment strength ($y_{\text{ali}}$) and the core alignment distance ($d0_{\text{ali}}$). If the GA selects a high gain for heading alignment, it must simultaneously widen the geometric zone where this force applies, preventing high-frequency heading oscillations that would destroy the smooth circular flow.
- **The Heading vs. Velocity Priority Trade-off ($r = -0.94$):** There is a severe negative correlation between the acceleration length scale ($l_{\text{acc}}$) and the alignment strength ($y_{\text{ali}}$). The GA identified that the swarm cannot rely heavily on both heading alignment and distant velocity synchronization simultaneously. If the swarm prioritizes matching headings, it must restrict the radius of speed adjustments to prevent conflicting command inputs that could fracture the swarm.

Ultimately, this parameter sweep demonstrates that the scalability of the bio-inspired swarm does not rely on a single, rigid set of parameters, but rather on a multidimensional manifold of ratios. By identifying these strict relational constraints, the genetic programming approach validated its efficiency in optimizing distributed flight dynamics.

## V. Application to Exploration and SAR Scenarios

### A. The Grid Coverage Problem

In Search and Rescue (SAR) operations, the primary objective translates mathematically to an area coverage optimization problem. The operational space is discretized into a 2D grid. The swarm's performance is intrinsically linked to its ability to explore the grid completely. Another important aspect of SAR scenarios is that the target might not stay in the same place during the whole scenario. This means that drones must overfly each cell of the grid repeatedly to ensure they extract as much information as possible from the environment.

### B. Fitness Evaluation for Exploration

The exploration fitness relies on two main factors: the continuous discovery of unseen areas and the periodic revisiting of previously explored zones. This mimics the decay of information relevant in dynamic SAR environments. The amount of information known about the cell is described by a numerical value called freshness. At the beginning of the scenario, each cell has a minimal freshness. As drones traverse the environment, they increase the freshness of the cells they overfly. To represent the information going stale over time, the freshness decreases (at first slowly, then faster as time passes).

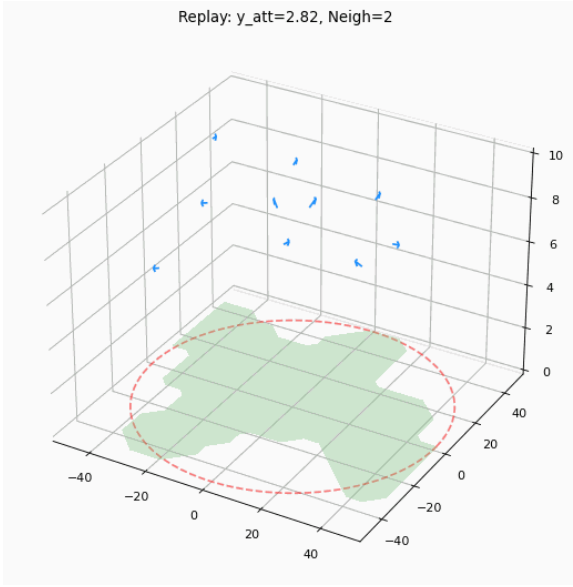



Fig. 4. Heatmap demonstrating the spatial coverage of the drone swarm operating under GA-optimized parameters.

The quality of the information gathered from a flight at low altitude is naturally better than that of the information gathered from a high altitude flight. However flying at higher altitude means the camera can see a wider area and thus gather more information. This relationship is taken into account by the fitness calculation. Each drone can "see" in a cone below them. If they fly high, they increase the freshness of a large amount of cells by a small amount. If they fly lower, they increase the freshness of a few cell by a larger amount.

The coverage of the grid is measured by counting the number of cells which are deemed "explored". For this to be the case the cell's freshness must exceed 10 % of the maximum freshness value. The coverage is then normalised by the total amount of cells in the grid. In the case where each drone has its own map of the grid (see section VI), a global grid is constructed with the best freshness found a cross the individual grids. This global grid is then used to calculate the coverage. The exploration cost $c_{\text{explo}}$ is this measure of the grid's coverage.

## VI. Distributed Information Gathering Strategies

A genetic algorithm can only be as good as its fitness function. In the case of a SAR scenario the fitness depends a lot on how the gathered information is shared between the drones in the swarm.

### A. Pheromone-based Stigmergic Coordination

One such method is pheromone-based stigmergic coordination [9], [10]. This method relies on digital pheromones being released by the drones to mark the cells they have explored. These pheromones decay and disappear over time, representing the information losing relevance as time goes on. The pheromones repel other drones, thus encouraging the swarm to fly to cells which have yet to be explored.

Xuan et al. [10] have even explored how the use of two distinct pheromones in an exploration scenario: a short term pheromone to encode the recent activity of the swarm and a long term pheromone to encode the global exploration coverage. They have showed this method can achieve a very high coverage uniformity of the water column.

In our simulation model, this stigmergic coordination is mathematically inverted and implemented through a discrete spatial "spoilage grid". Instead of maximizing freshness, the grid tracks spoilage, starting at a maximum value $S_{max}$ representing completely unexplored territory. As agents traverse the environment, they clear this spoilage. To account for altitude and sensor mechanics, the update is not a simple cell overwrite but a 2D Gaussian splat governed by the Inverse Square Law. The intensity and spread of the clearing effect dynamically depend on the drone's altitude ($Z$) and its Field of View (FOV).

To simulate the decaying relevance of information over time, the grid undergoes a uniform time-step spoilage accumulation governed by a multiplicative factor $\mu_{spoil}$ and an additive factor $\alpha_{spoil}$:

$$S_{t+1}(x, y) = \min(S_{max}, S_t(x, y) \cdot \mu_{spoil} + \alpha_{spoil})$$

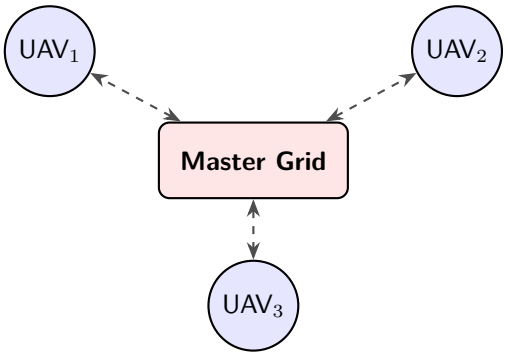
This increasing spoilage acts as an attractive virtual pheromone. Agents compute an exploration gradient vector directing them towards high-spoilage areas, driving the swarm naturally to continuously revisit and explore regions.

### B. Impact of Communication Topologies

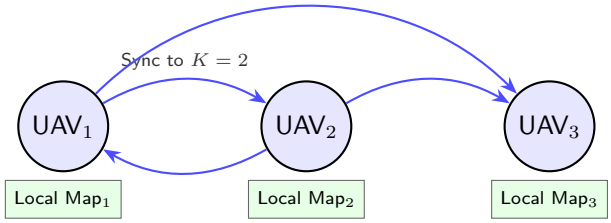
To evaluate the sim-to-real gap regarding network constraints, we compare several information-sharing topologies. The exploration framework relies on two distinct but complementary mechanisms: the map storage and sharing strategy (`MAP_STRATEGY`), and the navigation decision-making (`EXPLO_STRATEGY`).

While a centralized oracle provides theoretical maximum efficiency by granting all agents instantaneous access to a master freshness grid, real-world UAVs rely on distributed mesh networks with limited bandwidth and range. To accurately model these constraints, we implemented and evaluated three distinct communication paradigms for map synchronization:

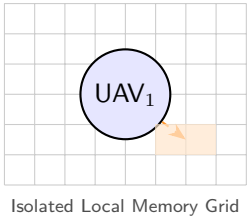
- **Global Oracle (`global`):** Agents have perfect, real-time knowledge of the entire environment's map. Every drone updates and reads from a single, centralized tensor. This serves as our theoretical upper-bound baseline.




- **Local Shared Map (`local_shared`):** Agents maintain individual, localized versions of the map. Synchronization occurs between drones using a topological approach rather than a strictly spatial one. Specifically, map data is shared unilaterally with an agent's $K$-Nearest Neighbors (e.g., $K = 2$). Furthermore, to simulate bandwidth constraints, this map synchronization is not continuous but occurs at a discrete interval of $\tau_{sync}$ ticks (e.g., `REFRESH_MAP_TICKS` $= 50$).




- **Local Individual Map (`local_individual`):** To severely reduce communication overhead and eliminate structural dependencies, drones do not exchange any map data over the network. Instead, each drone relies strictly on its own isolated memory grid, which is updated solely by its own field of view (FOV).




### C. Experimental Results and Comparative Analysis

To evaluate these topologies, we conducted a systematic parameter sweep crossing the mapping strategies with two decision policies: `local_gradient` (gradient descent on spoilage) and `global_best` (greedy targeting of the absolute maximum). The cost function minimizes $C_{total}$ with weights $W_{\text{coll}} = 500$ (collision penalty) and $W_{\text{explo}} = -50$ (coverage reward).

*1) Analysis of the Isolation Paradox:* The results in Table I reveal that the highest raw coverage (**93.35%**) is achieved in total isolation (`local_individual`). However,

TABLE I
PERFORMANCE METRICS ACROSS COMMUNICATION AND DECISION STRATEGIES.

| Map Strategy | Decision Policy | Total Cost ↓ | Coverage (%) ↑ |
|---|---|---|---|
| `global` | `global_best` | **-134,454** | 89.56% |
| `global` | `local_gradient` | -133,718 | 91.14% |
| `local_shared` | `global_best` | -130,901 | 87.97% |
| `local_shared` | `local_gradient` | -127,638 | 89.56% |
| `local_individual` | `global_best` | -121,514 | **93.35%** |
| `local_individual` | `local_gradient` | -120,901 | 71.52% |

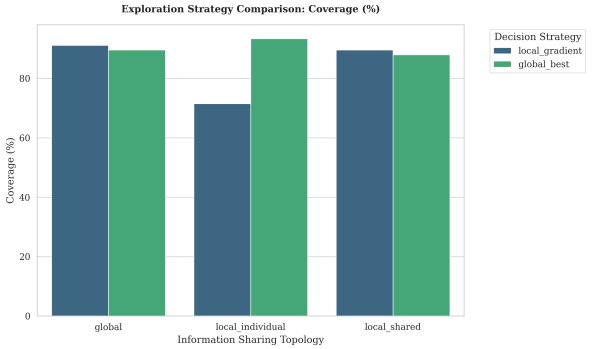



Fig. 5. Comparison of area coverage across topologies. Distributed sharing (`local_shared`) maintains efficiency close to the global oracle despite bandwidth constraints.

this configuration yields the worst global cost (-121,514). Without communication, drones target global maxima independently, leading to a "chaotic explosion" of the swarm. While this covers more area, the lack of coordination causes frequent trajectory intersections. Given that $W_{\text{coll}}$ is ten times higher than $W_{\text{explo}}$, the resulting collision penalties negate the exploration gains.

*2) Network Resilience and Bio-inspired Fluidity:* The `local_shared` topology achieves **89.56%** coverage, performing remarkably close to the global oracle. Coupled with `local_gradient`, the swarm maintains a fluid, ameba-like motion that minimizes collision risks. This validates that asynchronous, low-bandwidth sharing of localized maps is sufficient to successfully propagate stigmergic information and maintain efficient Search and Rescue operations in network-constrained environments. Indeed, comparable performance is achieved between the two map-sharing configurations, despite the distributed approach requiring less than a tenth of the network bandwidth

## VII. Discussion and Conclusion

The optimization of bio-inspired swarm parameters through genetic programming presents significant advantages for tailoring complex multi-agent systems to specific operational requirements. By leveraging PyTorch tensor operations, our framework massively parallelized the simulation environments, enabling the rapid and simultaneous evaluation of thousands of swarm configurations. Through the optimization of a multi-objective fitness function, the Genetic Algorithm successfully isolated parameter combinations that balance robust milling behavior, efficient area coverage, and minimal control effort.

A primary contribution of this study is the integration of realistic communication constraints into the evaluation pipeline. The implementation of discrete exploration modes, ranging from a centralized global oracle to decentralized local gradients and delayed map sharing, demonstrates how network limitations directly impact emergent swarm behaviors and global coverage efficiency. Furthermore, our parameter sweeping methodology highlighted the swarm's scalability challenges, quantitatively mapping how parameters optimized for a specific number of agents behave when deployed across varying swarm sizes.

Future work must focus on further narrowing the sim-to-real gap. While the current physics engine successfully incorporates inter-agent collision avoidance and basic kinematic updates, transitioning to a fully dynamic quadrotor simulation is the next critical step. This necessitates the introduction of sensory noise, complex communication packet loss models, and high-fidelity aerodynamic constraints. Additionally, integrating dynamic obstacles and individual battery life constraints will be essential to validate the robustness of these bio-inspired control algorithms for real-world Search and Rescue deployments.

## References


[1] M. Verdoucq, "Contrôle bio-inspiré des mouvements collectifs d'un essaim de drones et applications à des scénarios opérationnel," Ph.D. dissertation, Université de Toulouse, Toulouse, Feb. 2024.

[2] M. Verdoucq, G. Theraulaz, Ramon Escobedo, C. Sire, and G. Hattenberger, "Bio-inspired control for collective motion in swarms of drones," in *2022 International Conference on Unmanned Aircraft Systems (ICUAS)*. Dubrovnik, Croatia: IEEE, Jun. 2022, pp. 1626–1631. [Online]. Available: https://ieeexplore.ieee.org/document/9836112/

[3] T. Vicsek and A. Zafeiris, "Collective motion," *Physics Reports*, vol. 517, no. 3-4, pp. 71–140, Aug. 2012, arXiv:1010.5017 [cond-mat]. [Online]. Available: http://arxiv.org/abs/1010.5017

[4] L. Lei, R. Escobedo, C. Sire, and G. Theraulaz, "Computational and robotic modeling reveal parsimonious combinations of interactions between individuals in schooling fish," *PLOS Computational Biology*, vol. 16, no. 3, p. e1007194, Mar. 2020. [Online]. Available: https://dx.plos.org/10.1371/journal.pcbi.1007194

[5] M. Verdoucq, D. Trendafilov, C. Sire, R. Escobedo, G. Theraulaz, and G. Hattenberger, "Flocking phase transition and threat responses in bio-inspired autonomous drone swarms," Dec. 2025, arXiv:2512.21196 [cs]. [Online]. Available: http://arxiv.org/abs/2512.21196

[6] R. Poli, W. Langdon, and N. Mcphee, *A Field Guide to Genetic Programming*, Jan. 2008.

[7] S. Arbaç and T. V. Mumcu, "Multi-Objective Genetic Algorithm Framework in Swarm UAV Systems with Real-Time Simulation," *IFAC-PapersOnLine*, vol. 59, no. 18, pp. 169–174, 2025. [Online]. Available: https://linkinghub.elsevier.com/retrieve/pii/S2405896325016143

[8] A. Neupane and M. Goodrich, "Learning Swarm Behaviors using Grammatical Evolution and Behavior Trees," Aug. 2019, pp. 513–520.

[9] S. Devaraju, A. Ihler, and S. Kumar, "Connectivity-Aware Pheromone Mobility Model for Autonomous UAV Networks," in *2023 IEEE 20th Consumer Communications & Networking Conference (CCNC)*, Jan. 2023, pp. 1–6, arXiv:2210.06684 [cs]. [Online]. Available: http://arxiv.org/abs/2210.06684

[10] L. Xuan, M. Liu, G. He, and Z. Yan, "Dual-Trail Stigmergic Coordination Enables Robust Three-Dimensional Underwater Swarm Coverage," *Journal of Marine Science and Engineering*, vol. 14, no. 2, p. 164, Jan. 2026. [Online]. Available: https://www.mdpi.com/2077-1312/14/2/164